



Instituto Tecnológico de Costa Rica
Escuela de Ingeniería en Computadores
CE1103 - Algoritmos y Estructuras de Datos I

Proyecto #1: MESSAGE QUEUE

Samuel Ricardo Morales Cascante - 2024130487
Antony Javier Hernández Castillo - 2022321746

s.morales.2@estudiantec.cr
anthernandez@estudiantec.cr

Cartago, Costa Rica
I Semestre 2025

Profesor:
Ing. Isaac Ramírez, M.SI.

Índice

I	Introducción	2
II	Descripción del Problema	2
III	Descripción de la solución	2
III-A	Primera parte: MQBroker	3
III-A1	Requerimiento 001:	3
III-A2	Requerimiento 002:	7
III-A3	Requerimiento 003:	9
III-A4	Requerimiento 004:	9
III-A5	Requerimiento 005:	10
III-B	Segunda parte: MQClient	11
III-B1	Requerimiento 001: MQClient(string ip, int port, Guid AppID)	12
III-B2	Requerimiento 002: bool Subscribe(Topic topic)	13
III-B3	Requerimiento 003: bool Unsubscribe(Topic topic)	15
III-B4	Requerimiento 004: bool Publish(Message message, Topic topic)	16
III-B5	Requerimiento 005: Message Receive(Topic topic)	17
III-C	Tercera Parte: Interfaz Gráfica en .NET Core	18
IV	Diseño General	20
v	Link al repositorio del proyecto en Github	20
	Referencias	20

I. INTRODUCCIÓN

Los sistemas distribuidos modernos requieren mecanismos eficientes y escalables para la comunicación asíncrona entre componentes, especialmente en entornos donde la disponibilidad y el desacoplamiento son críticos [1]. Las colas de mensajes (Message Queues) han surgido como una solución fundamental para este desafío, permitiendo la transmisión de datos entre aplicaciones independientes sin acoplamiento temporal o espacial [2]. Este trabajo presenta el diseño e implementación de un middleware de colas de mensajes basado en el modelo publish/subscribe, que aborda los problemas de escalabilidad y robustez en sistemas distribuidos mediante estructuras de datos lineales personalizadas y un protocolo de comunicación ligero. La arquitectura propuesta sigue el patrón de intermediación (broker), donde un servidor central gestiona suscripciones, publicación y entrega de mensajes bajo el principio FIFO (First-In-First-Out). Este enfoque contrasta con soluciones tradicionales como RabbitMQ, que, aunque robusta, introduce complejidad y overhead en escenarios de pequeña a mediana escala. Nuestra implementación demuestra que es posible lograr un equilibrio entre simplicidad y funcionalidad mediante el uso de estructuras de datos optimizadas (listas, colas y diccionarios) y sockets TCP para la comunicación [3]. Además, este trabajo contribuye con:

1. Un análisis comparativo de estructuras de datos lineales para manejo de colas en tiempo real.
2. Un protocolo de mensajería minimalista que reduce la latencia en comparación con alternativas basadas en HTTP/REST [4].
3. Una biblioteca cliente reutilizable que abstrae la complejidad de la comunicación distribuida.

Los resultados validan la eficacia del sistema en escenarios con alta concurrencia de mensajes, destacando su potencial para aplicaciones en IoT, microservicios y sistemas de procesamiento de eventos [5]. El código fuente y las métricas de rendimiento están disponibles para su revisión.

II. DESCRIPCIÓN DEL PROBLEMA

En sistemas distribuidos, la comunicación entre aplicaciones independientes es un desafío crítico, especialmente cuando se requiere desacoplamiento temporal y espacial. Las soluciones tradicionales, como las APIs REST sincrónicas, presentan limitaciones en escalabilidad y tolerancia a fallos, ya que exigen que los servicios estén disponibles simultáneamente y pueden generar cuellos de botella en entornos de alta concurrencia.

Este proyecto aborda este problema mediante la implementación de un middleware de colas de mensajes (Message Queue Broker) basado en el modelo publish/subscribe, que permite:

- Comunicación asíncrona: Los productores y consumidores de mensajes no necesitan estar activos al mismo tiempo.
- Escalabilidad: El broker gestiona múltiples colas de mensajes, distribuyendo la carga eficientemente.
- Desacoplamiento: Las aplicaciones solo interactúan con el broker, sin necesidad de conocer detalles de implementación entre sí.

III. DESCRIPCIÓN DE LA SOLUCIÓN

Se propone un Message Queue Broker basado en el modelo publish/subscribe, implementado con:

1. Estructuras de datos optimizadas:
 - Colas personalizadas (FIFO) para garantizar el orden de los mensajes.
 - Diccionarios anidados para gestionar suscripciones por tema (topic) y aplicación (AppID).
2. Comunicación mediante sockets TCP:
 - Protocolo ligero con formato COMANDO—PARÁMETROS para minimizar overhead.

- Manejo de concurrencia con locks y ThreadPool para evitar condiciones de carrera.
3. Robustez frente a fallos:
- Validación de suscripciones antes de entregar mensajes.
 - Respuestas de error estandarizadas para clientes (ej.: ERROR—No hay mensajes).

La solución demuestra que es posible construir un sistema ligero y eficiente utilizando estructuras personalizadas, ofreciendo una alternativa viable para entornos donde soluciones empresariales resultan excesivas.

A continuación, se abordará detalladamente la implementación utilizada en cada requerimiento del proyecto.

III-A. Primera parte: MQBroker

La primera parte es un programa que deberá crear llamado MQBroker. Es un proyecto consola en .NET que se encarga de gestionar las colas y transmitir los mensajes entre las colas. MQBroker utiliza una topología publicador/subscriptor. MQBroker utiliza un socket servidor para escuchar las peticiones de las aplicaciones que envían/consumen mensajes. A continuación se listan los requerimientos para este programa y cómo se implementaron:

III-A1. Requerimiento 001:

Al iniciar, se activa un socket para escuchar peticiones entrantes de los clientes (aplicaciones que envían/reciben mensajes). El formato de los mensajes será definido por los estudiantes. Debe tener un formato consistente para todas las peticiones.

La solución implementada utiliza un broker de mensajes básico que escucha peticiones entrantes de clientes a través de sockets TCP. El broker permite cuatro operaciones principales: suscripción a tópicos, desuscripción, publicación de mensajes y recepción de mensajes.

III-A1a. Estructura del Código

1. Estructuras de Datos Personalizadas: Antes de implementar el broker, se crearon estructuras de datos personalizadas para manejar las colecciones necesarias:
 - **MyList<T>- Lista Genérica**

```
1 public class MyList<T>
2 {
3     private T[] items;
4     private int count;
5     private int capacity;
6
7     public MyList(int initialCapacity = 4)
8     {
9         capacity = initialCapacity;
10        items = new T[capacity];
11        count = 0;
12    }
13
14    public void Add(T item)
15    {
16        if (count == capacity)
17        {
18            // Duplicar capacidad si es necesario
19            capacity *= 2;
20            T[] newItems = new T[capacity];
21            Array.Copy(items, newItems, count);
22            items = newItems;
23        }
24        items[count] = item;
25        count++;
```

```
26     }
27 }
```

Listing 1: Ejemplo de clase en C#

El propósito de la implementación del listing 1 es el de proporcionar una lista dinámica con funcionalidad similar a `List<T>` pero implementada manualmente para comprender el comportamiento de este.

■ `MyQueue<T>`- Cola Personalizada

Para la implementación de la clase `MyQueue` se hace uso de una cola FIFO (First in, first out) para el almacenamiento de mensajes pendientes para cada subscriptor.

```
1 class MyQueue<T>
2 {
3     private T[] items;
4     private int head;
5     private int tail;
6     private int count;
7     private int capacity;
8
9     public MyQueue(int initialCapacity = 4)
10    {
11        capacity = initialCapacity;
12        items = new T[capacity];
13        head = 0;
14        tail = 0;
15        count = 0;
16    }
17
18    public int Count => count;
19
20    public void Enqueue(T item)
21    {
22        if (count == capacity)
23        {
24            int newCapacity = capacity * 2;
25            T[] newItems = new T[newCapacity];
26            for (int i = 0; i < count; i++)
27            {
28                newItems[i] = items[(head + i) % capacity];
29            }
30            items = newItems;
31            head = 0;
32            tail = count;
33            capacity = newCapacity;
34        }
35
36        items[tail] = item;
37        tail = (tail + 1) % capacity;
38        count++;
39    }
40 }
```

Listing 2: Cola personalizada

■ `MyDictionary <TKey, TValue T>`- Diccionario Personalizado Se implementa un diccionario personalizado para almacenar relaciones clave-valor para así gestionar subscriptores y colas de mensajes.

```
1 class MyDictionary<TKey, TValue>
```

```

2 {
3     private struct KeyValuePair
4     {
5         public TKey Key;
6         public TValue Value;
7     }
8
9     private MyList<KeyValuePair> items;
10
11     public MyDictionary()
12     {
13         items = new MyList<KeyValuePair>();
14     }
15
16     public void Add(TKey key, TValue value)
17     {
18         if (ContainsKey(key))
19         {
20             throw new ArgumentException("La clave ya existe");
21         }
22         items.Add(new KeyValuePair { Key = key, Value = value });
23     }
24
25     public bool ContainsKey(TKey key)
26     {
27         for (int i = 0; i < items.Count; i++)
28         {
29             if (EqualityComparer<TKey>.Default.Equals(items[i].Key, key))
30             {
31                 return true;
32             }
33         }
34         return false;
35     }
36 }

```

Listing 3: Diccionario personalizado

2. Clase Principal MQBroker

```

1
2 class MQBroker
3 {
4     private TcpListener server;
5     private MyDictionary<string, MyList<Guid>> suscriptores;
6     private MyDictionary<string, MyDictionary<Guid, MyQueue<string>>> colasMensajes;
7
8     public MQBroker(string ip, int port)
9     {
10         server = new TcpListener(IPAddress.Parse(ip), port);
11         suscriptores = new MyDictionary<string, MyList<Guid>>();
12         colasMensajes = new MyDictionary<string, MyDictionary<Guid, MyQueue<string>>>();
13     }
14 }

```

Listing 4: Configuración inicial y declaración de objetos de la clase MQBroker

- **Inicialización del Servidor** El código que controla la inicialización del servidor es el siguiente:

```

1
2 public void Start()
3 {
4     server.Start();

```

```

5 Console.WriteLine("MQBroker escuchando en {0}", server.LocalEndpoint);
6
7 try
8 {
9     while (true)
10    {
11        TcpClient client = server.AcceptTcpClient();
12        ThreadPool.QueueUserWorkItem(state => HandleClient((TcpClient)state),
client);
13    }
14 }
15 catch (Exception ex)
16 {
17     Console.WriteLine("Error en el servidor: " + ex.Message);
18 }
19 finally
20 {
21     server.Stop();
22 }
23 }

```

Listing 5: Inicialización del Servidor

Su funcionamiento e implementación se basa en lo siguiente:

- a) Inicia el TcpListener en la IP y puerto especificados.
- b) Entra en un bucle infinito aceptando conexiones.
- c) Para cada cliente, delega el manejo a un hilo del ThreadPool.

■ Manejo de Clientes

```

1
2 private void HandleClient(TcpClient client)
3 {
4     try
5     {
6         using (NetworkStream stream = client.GetStream())
7         {
8             byte[] buffer = new byte[4096];
9             int bytesRead = stream.Read(buffer, 0, buffer.Length);
10            string request = Encoding.UTF8.GetString(buffer, 0, bytesRead);
11
12            Console.WriteLine($"Recibido: {request}");
13
14            string response = ProcesarPeticion(request);
15            byte[] responseBytes = Encoding.UTF8.GetBytes(response);
16
17            stream.Write(responseBytes, 0, responseBytes.Length);
18        }
19    }
20    catch (Exception ex)
21    {
22        Console.WriteLine("Error al manejar el cliente: " + ex.Message);
23    }
24    finally
25    {
26        client.Close();
27    }
28 }

```

Listing 6: Método del manejo de Clientes

El flujo del manejo de clientes sigue la siguiente secuencia:

- a) Lee los datos del socket (hasta 4KB).
- b) Convierte los bytes a string UTF-8.
- c) Procesa la petición.
- d) Envía la respuesta al cliente.

3. Formato de Mensajes y Procesamiento

- El formato de mensaje que se definió es el siguiente:

```
1
2 COMANDO|PAR METRO1|PAR METRO2|...|PAR METRON
```

Listing 7: Formato de mensaje

- Procesamiento de Peticiones

```
1
2 private string ProcesarPetición(string request)
3 {
4     try
5     {
6         string[] partes = request.Split('|');
7
8         if (partes.Length < 2)
9         {
10             return "ERROR|Formato de petición inválido";
11         }
12
13         string comando = partes[0];
14
15         switch (comando)
16         {
17             case "SUBSCRIBE":
18                 return Subscribe(partes[1], Guid.Parse(partes[2]));
19             case "UNSUBSCRIBE":
20                 return Unsubscribe(partes[1], Guid.Parse(partes[2]));
21             case "PUBLISH":
22                 return Publish(partes[1], partes[2], Guid.Parse(partes[3]));
23             case "RECEIVE":
24                 return Receive(partes[1], Guid.Parse(partes[2]));
25             default:
26                 return "ERROR|Comando desconocido: " + comando;
27         }
28     }
29     catch (Exception ex)
30     {
31         return $"ERROR|Error interno: {ex.Message}";
32     }
33 }
```

Listing 8: Procesamiento de Peticiones del MQBroker

III-A2. Requerimiento 002:

Escucha una petición llamada Subscribe. La petición incluye:

- AppID en formato GUID generador por la aplicación que envía la petición.
- Tema al que se quiere subscribir.

Al recibir esta petición, MQBroker verifica si dicho AppID está o no suscrito al tema. Si está suscrito, retorna un mensaje que lo indique. Si no, crea una cola exclusiva para dicho tema y dicho AppID.

IMPLEMENTACIÓN PARA EL REQUERIMIENTO 002 DEL MQBROKER

El sistema gestiona las suscripciones mediante un diccionario anidado, donde cada topic (clave principal) contiene una lista de AppIDs suscritos. La estructura central se define así:

```
1 // Estructuras de datos para manejar suscripciones y colas
2 private MyDictionary<string, MyList<Guid>> suscriptores; // Tema - Lista de AppIDs
3 private MyDictionary<string, MyDictionary<Guid, MyQueue<string>>> colasMensajes; // Tema - (
4 AppID - Cola de mensajes)
```

Listing 9: Manejo de suscripciones y colas

Al recibir una petición SUBSCRIBE—tema—AppID, el broker ejecuta el siguiente flujo:

```
1 private string Subscribe(string tema, Guid appId)
2 {
3     lock (suscriptores) // Bloqueo para evitar condiciones de carrera
4     {
5         // 1. Verificar si el tema existe; si no, crearlo
6         if (!suscriptores.ContainsKey(tema))
7         {
8             suscriptores[tema] = new MyList<Guid>();
9         }
10
11         // 2. Validar si ya est suscrito
12         if (suscriptores[tema].Contains(appId))
13         {
14             return "ERROR|Ya est suscrito al tema: " + tema;
15         }
16
17         // 3. Registrar suscriptor y crear su cola exclusiva
18         suscriptores[tema].Add(appId);
19
20         if (!colasMensajes.ContainsKey(tema))
21         {
22             colasMensajes[tema] = new MyDictionary<Guid, MyQueue<string>>();
23         }
24         colasMensajes[tema][appId] = new MyQueue<string>(); // Cola FIFO vac a
25
26         return "OK|Suscripci n exitosa";
27     }
28 }
29 }
```

Listing 10: Método para que el cliente se suscriba al tema seleccionado

Se evaluó un manejo jerárquico de temas. Para soportar temas como Tema/Noticias/*, se consideró un árbol de prefijos, pero aumentaba la complejidad del proyecto y no estaba dentro del alcance de este. Un ejemplo de cómo se pudo haber implementado en el proyecto es:

```
1 // Ejemplo de estructura para wildcards (no implementada)
2 class TopicNode
3 {
4     public MyDictionary<string, TopicNode> Children;
5     public MyList<Guid> Subscribers;
6 }
7 }
```

Listing 11: Ejemplo de uso de árbol de prefijos para la clasificación de temas

III-A3. Requerimiento 003:

Escucha una petición llamada Unsubscribe. La petición incluye:

- AppID en formato GUID generador por la aplicación que envía la petición.
- Tema al que se quiere suscribir.

Al recibir esta petición, MQBroker verifica si dicho AppID está o no suscrito al tema. Si no está suscrito, retorna un mensaje que lo indique. Si no, elimina la cola para dicho tema y dicho AppId.

IMPLEMENTACIÓN PARA EL REQUERIMIENTO 003 DEL MQBROKER

El sistema maneja las desuscripciones mediante el mismo diccionario anidado usado para las suscripciones. Al recibir una petición UNSUBSCRIBE—tema—AppID, el broker ejecuta el siguiente flujo:

```
1 private string Unsubscribe(string tema, Guid appId)
2 {
3     lock (suscriptores)
4     {
5         // Verificar si el tema existe y el AppID est suscrito
6         if (!suscriptores.ContainsKey(tema) || !suscriptores[tema].Contains(appId))
7         {
8             return "ERROR|No est suscrito al tema: " + tema;
9         }
10
11         // Eliminar de la lista de suscriptores
12         suscriptores[tema].Remove(appId);
13
14         // Eliminar la cola de mensajes
15         if (colasMensajes.ContainsKey(tema))
16         {
17             MyDictionary<Guid, MyQueue<string>> colas = colasMensajes[tema];
18             if (colas.ContainsKey(appId))
19             {
20                 colas.Remove(appId);
21             }
22         }
23
24         Console.WriteLine($"Desuscripci3n exitosa: {appId} de {tema}");
25         return "OK|Desuscripci3n exitosa";
26     }
27 }
28 }
```

Listing 12: Método Unsubscribe del MQBroker

Una limitación encontrada de esta implementación es que, al desuscribirse, los mensajes no consumidos en la cola se pierden permanentemente. Una mejora sería implementar un mecanismo de retención temporal.

III-A4. Requerimiento 004:

La petición Publish incluye:

- AppID en formato GUID de la aplicación que publica el mensaje.
- Tema al que se desea publicar en formato string. Por ejemplo: Tema/TipoDeCambio/Dolares.
- Contenido de la publicación, es decir el cuerpo del mensaje como tal.

Cuando el broker recibe una petición publish, verifica que el tema exista. Si no existe, ignora la publicación. Si el tema existe, por cada uno de los suscriptores para dicho tema, coloca un mensaje en su cola respectiva.

IMPLEMENTACIÓN PARA EL REQUERIMIENTO 004 DEL MQBROKER

El sistema procesa las publicaciones mediante el siguiente flujo, utilizando las estructuras de datos de suscripciones y colas previamente definidas:

```
1 private string Publish(string tema, string mensaje, Guid appId)
2 {
3     lock (suscriptores)
4     {
5         // Verificar si el tema existe
6         if (!suscriptores.ContainsKey(tema) || suscriptores[tema].Count == 0)
7         {
8             Console.WriteLine($"Mensaje ignorado - Tema: {tema}");
9             return "ERROR|No hay suscriptores para este tema: " + tema;
10        }
11
12        // Asegurarse de que existe el diccionario de colas para este tema
13        if (!colasMensajes.ContainsKey(tema))
14        {
15            return "ERROR|Tema no configurado correctamente: " + tema;
16        }
17
18        int mensajesEnviados = 0;
19        MyDictionary<Guid, MyQueue<string>> colas = colasMensajes[tema];
20
21        // Para cada suscriptor, colocar el mensaje en su cola
22        for (int i = 0; i < suscriptores[tema].Count; i++)
23        {
24            Guid subscriber = suscriptores[tema][i];
25
26            if (colas.ContainsKey(subscriber))
27            {
28                colas[subscriber].Enqueue(mensaje);
29                mensajesEnviados++;
30            }
31        }
32
33        Console.WriteLine($"Mensaje publicado en {tema}: {mensaje.Substring(0, Math.Min(20,
34        mensaje.Length))}...");
35        return $"OK|Mensaje publicado a {mensajesEnviados} suscriptores";
36    }
37 }
```

Listing 13: Método Publish del MQBroker

El método Publish se encarga de distribuir un mensaje a todos los suscriptores registrados en un tema específico, utilizando un sistema de colas para garantizar el envío ordenado. Funciona bajo un bloqueo (lock) para asegurar la consistencia en entornos multihilo, evitando modificaciones concurrentes en las estructuras de datos compartidas (suscriptores y colasMensajes).

III-A5. Requerimiento 005:

La petición Receive permite a una aplicación específica, obtener un mensaje de la cola. Es decir, los mensajes colocados en la cola por el broker, se quedan en la cola hasta que la aplicación correspondiente solicite obtener dicho mensaje. La petición receive incluye:

- AppID en formato GUID de la aplicación que desea obtener el mensaje y que previamente se suscripción.
- Tema en formato string del que desea obtener un mensaje.

Al recibir la petición, el broker verifica que el AppID esté suscrito a dicho mensaje, si no lo está retorna un error. De lo contrario, verifica si hay algún mensaje en la cola de dicho AppID para ese tema. Si no

hay mensaje, retorna un error. Si hay un mensaje, lo saca de la cola (FIFO) y lo retorna a la aplicación a través del socket.

IMPLEMENTACIÓN PARA EL REQUERIMIENTO 005 DEL MQBROKER

El método Receive permite a una aplicación suscrita (appId) recuperar el siguiente mensaje disponible en un tema específico, siguiendo un enfoque FIFO (First-In-First-Out). Al igual que Publish, opera bajo un bloqueo (lock) para garantizar la seguridad en hilos (thread-safe), evitando condiciones de carrera al acceder a las estructuras compartidas (suscriptores y colasMensajes). Se implementó de la siguiente manera:

```
1
2 private string Receive(string tema, Guid appId)
3 {
4     lock (suscriptores)
5     {
6         // Verificar si el AppID est suscrito al tema
7         if (!suscriptores.ContainsKey(tema) || !suscriptores[tema].Contains(appId))
8         {
9             Console.WriteLine($"No se encuentra suscrito al tema: {tema}");
10            return "ERROR|No est suscrito al tema: " + tema;
11        }
12
13        // Verificar si existe la cola y tiene mensajes
14        if (!colasMensajes.ContainsKey(tema) || !colasMensajes[tema].ContainsKey(appId))
15        {
16            Console.WriteLine($"Mensaje no encontrado - Tema: {tema}");
17            return "ERROR|Cola no encontrada";
18        }
19
20        MyQueue<string> cola = colasMensajes[tema][appId];
21
22        if (cola.Count == 0)
23        {
24            Console.WriteLine($"No existen mensajes en cola del tema: {tema}");
25            return "ERROR|No hay mensajes en la cola";
26        }
27
28        // Sacar un mensaje de la cola (FIFO)
29        string mensaje = cola.Dequeue();
30
31        Console.WriteLine($"Mensaje entregado a {appId} desde {tema}");
32        return $"OK|{mensaje}";
33    }
34 }
```

Listing 14: Método Recieve del MQBroker

Este método complementa el sistema Pub/Sub, permitiendo a los suscriptores consumir mensajes de manera ordenada y bajo exclusión mutua. La separación entre Publish (producción) y Receive (consumo) asegura un flujo de datos consistente, ideal para escenarios asincrónicos o basados en eventos

III-B. Segunda parte: MQClient

La segunda parte es una biblioteca (class library) en .NET que permite a los clientes, interactuar con el broker. Contiene la clase facade MQClient. Dicha clase contiene al menos los siguientes métodos públicos:

III-B1. Requerimiento 001: MQClient(string ip, int port, Guid AppID)

Constructor que crea un nuevo MQClient. Recibe el IP y el puerto donde está escuchando el MQBroker. Además recibe el AppID (generado por la aplicación que utiliza la biblioteca MQClient). Este método puede realizar cualquier inicialización requerida por la biblioteca.

IMPLEMENTACIÓN PARA EL REQUERIMIENTO 001 DEL MQCLIENT

La clase MQClient es el núcleo de una biblioteca cliente para interactuar con un Message Queue Broker (MQBroker). El código se organiza con estos componentes principales.

1. Empezando por el constructor, el código clave del constructor es el siguiente:

```
1 public MQClient(string ip, int port, Guid appId)
2 {
3     if (string.IsNullOrEmpty(ip))
4         throw new ArgumentException("La dirección IP no puede estar vacía", nameof(ip));
5
6     if (port <= 0 || port > 65535)
7         throw new ArgumentException("El puerto debe estar entre 1 y 65535", nameof(port));
8
9     if (appId == Guid.Empty)
10         throw new ArgumentException("El AppID no puede ser vacío", nameof(appId));
11
12     brokerIp = ip;
13     brokerPort = port;
14     this.appId = appId;
15 }
16 }
```

Listing 15: Constructor clave del MQClient

Este tiene la responsabilidad de inicializar el cliente con los parámetros esenciales para conectarse al broker (ip, port, appId), además de que valida que la IP no esté vacía, esté dentro de un rango válido y que appId no sea Guid.Empty.

2. Clases de dominio (Topic, Message):
 - Topic: Valida y encapsula el nombre de un tema (ej: "noticias"). Lanza excepciones si el nombre es inválido.
 - Message: Representa el contenido de un mensaje, con métodos para parsear respuestas del broker (ej: FromResponse).
3. Excepciones personalizadas:
 - MQClientException (base) y derivadas (MQSubscriptionException, MQPublishException, MQReceiveException) para manejar errores específicos.
4. Clase MQClient:
 - Gestiona la conexión con el broker y ejecuta las operaciones.
5. Comunicación con el Broker El método privado SendRequest es el corazón de la comunicación:
 - a) Conexión TCP con timeouts:
 - Usa TcpClient con un timeout de 5 segundos para conexión y operaciones (ReadTimeout/WriteTimeout).
 - b) Protocolo de mensajes:
 - Envía comandos en formato texto (ej: "PUBLISH—tema—contenido—appId")
 - Espera respuestas con prefijos OK— o ERROR—.

El método que maneja la comunicación con el Broker está implementado de la siguiente manera:

```

1 private string SendRequest(string message)
2 {
3     using (TcpClient client = new TcpClient())
4     {
5         var result = client.BeginConnect(brokerIp, brokerPort, null, null);
6         bool success = result.AsyncWaitHandle.WaitOne(TimeSpan.FromSeconds(5));
7
8         if (!success)
9             throw new MQClientException("Timeout al conectar con el broker");
10
11         client.EndConnect(result);
12
13         using (NetworkStream stream = client.GetStream())
14         {
15             stream.ReadTimeout = 5000;
16             stream.WriteTimeout = 5000;
17
18             byte[] data = Encoding.UTF8.GetBytes(message);
19             stream.Write(data, 0, data.Length);
20
21             byte[] buffer = new byte[4096];
22             int bytesRead = stream.Read(buffer, 0, buffer.Length);
23             return Encoding.UTF8.GetString(buffer, 0, bytesRead);
24         }
25     }
26 }

```

Listing 16: Método SendRequest que permite la comunicación con el Broker

Esta implementación ofrece una API robusta y segura para interactuar con un broker de mensajería, priorizando:

- Validaciones tempranas.
- Manejo explícito de errores.
- Liberación adecuada de recursos.
- Claridad en el protocolo de comunicación (texto plano con prefijos).

Es por ello que esta implementación utilizada es adecuada para escenarios donde se requiere sencillez y confiabilidad en la comunicación cliente-broker.

III-B2. Requerimiento 002: bool Subscribe(Topic topic)

Envía la petición Subscribe al MQBroker mediante un socket de tipo cliente. Nótese que el argumento es un objeto tipo Topic. Este es un objeto definido por ud que encapsula un string que contiene el tema. La idea es que al utilizar un objeto de nivel más alto, se pueden realizar validaciones y otras modificaciones relacionadas con temas. Retorna true o false indicando si la suscripción se realizó o no. Debe lanzar excepciones relevantes según aplique.

IMPLEMENTACIÓN PARA EL REQUERIMIENTO 002 DEL MQCLIENT

El método Subscribe permite que un cliente se suscriba a un tema específico en el broker, utilizando un objeto de tipo Topic para garantizar que el nombre del tema sea válido. Su implementación se intentó que siga un enfoque robusto, manejando posibles errores mediante excepciones y asegurando una comunicación confiable con el broker mediante sockets TCP. Un flujo de acciones que debe seguir el método para lograr lo anteriormente dicho, se puede plantear como:

1. Validación del parámetro topic:
 - Verifica que el objeto Topic no sea null. Si lo es, lanza una ArgumentNullException.

- El propio constructor de Topic ya valida que el nombre no esté vacío (como se vio en el Requerimiento 001).

2. Construcción del mensaje de suscripción:

- Formato: "SUBSCRIBE—nombreTema—appId".
- Ejemplo: Si el tema es "noticias" el appId es "123e4567-e89b-12d3-a456-426614174000", el mensaje sería:

```
1  
2 SUBSCRIBE|noticias|123e4567-e89b-12d3-a456-426614174000
```

Listing 17: Impresión del mensaje de suscripción

3. Envío de la solicitud al broker:

- Usa el método privado SendRequest (definido previamente) para enviar el comando a través de un socket TCP.
- SendRequest maneja timeouts (5 segundos) y errores de conexión, lanzando MQClientException si falla.

4. Procesamiento de la respuesta:

- Respuesta exitosa (OK—): Retorna true.
- Respuesta de error (ERROR—):
 - Si el error es "Ya está suscrito", retorna true (para ser idempotente).
 - En otros casos, lanza una MQSubscriptionException con el mensaje de error.
- Respuesta inesperada: Lanza MQSubscriptionException.

La implementación del código del método que maneja las suscripciones es:

```
1  
2 public bool Subscribe(Topic topic)  
3 {  
4     if (topic == null)  
5     {  
6         throw new ArgumentNullException(nameof(topic), "El tema no puede ser nulo");  
7     }  
8  
9     try  
10    {  
11        string request = $"SUBSCRIBE|{topic.Name}|{appId}";  
12        string response = SendRequest(request);  
13  
14        if (response.StartsWith("OK|"))  
15        {  
16            return true;  
17        }  
18        else if (response.StartsWith("ERROR|"))  
19        {  
20            string errorMessage = response.Substring(6); // Skip "ERROR|"  
21            if (errorMessage.Contains("Ya est  suscrito"))  
22            {  
23                // Si ya est  suscrito, consideramos que la operaci n es exitosa  
24                return true;  
25            }  
26            throw new MQSubscriptionException(errorMessage);  
27        }  
28        else  
29        {  
30            throw new MQSubscriptionException("Respuesta inesperada del broker");  
31        }  
32    }  
33 }
```

```

33     catch (MQClientException ex) when (!(ex is MQSubscriptionException))
34     {
35         throw new MQSubscriptionException($"Error al suscribirse: {ex.Message}");
36     }
37 }

```

Listing 18: Implementación método Subscribe

III-B3. Requerimiento 003: *bool Unsubscribe(Topic topic)*

Envía la petición Unsubscribe al MQBroker mediante un socket de tipo cliente. Retorna true o false indicando si se pudo o no de suscribir del tema. Debe lanzar excepciones relevantes según aplique.

IMPLEMENTACIÓN PARA EL REQUERIMIENTO 003 DEL MQCLIENT

El método Unsubscribe permite que un cliente cancele su suscripción a un tema previamente suscrito en el broker. Al igual que Subscribe, utiliza un objeto Topic para garantizar la validez y sigue un protocolo estructurado para comunicarse con el broker mediante sockets TCP. Debe ser altamente retroalimentativo con el cliente, mostrando suficientes excepciones de error. El flujo que debe seguir el método se puede plantear como:

1. Validación del parámetro topic:
 - Rechaza topic nulo con ArgumentNullException.
 - La validación del nombre del tema ya se hace en el constructor de Topic.
2. Construcción del mensaje:
 - Formato: UNSUBSCRIBE—nombreTema—appId”.
3. Envío al broker mediante SendRequest:
 - Usa el mismo método auxiliar para TCP (con timeouts de 5 segundos).
4. Procesamiento de la respuesta:
 - OK—: Retorna true (éxito).
 - ERROR—:
 - Si el error es ”No está suscrito”, retorna true (idempotencia)
 - Otros errores lanzan MQSubscriptionException.
 - Respuestas inesperadas lanzan excepciones.

La implementación del código para el manejo de desuscripciones de los usuarios se realizó de la siguiente manera:

```

1
2 public bool Unsubscribe(Topic topic)
3 {
4     if (topic == null)
5     {
6         throw new ArgumentNullException(nameof(topic), "El tema no puede ser nulo");
7     }
8
9     try
10    {
11        string request = $"UNSUBSCRIBE|{topic.Name}|{appId}";
12        string response = SendRequest(request);
13
14        if (response.StartsWith("OK|"))
15        {
16            return true;
17        }
18    }
19 }

```



```

18     else if (response.StartsWith("ERROR|"))
19     {
20         string errorMessage = response.Substring(6); // Skip "ERROR|"
21         if (errorMessage.Contains("No est suscrito"))
22         {
23             // Si no est suscrito, consideramos que ya est en el estado deseado
24             return true;
25         }
26         throw new MQSubscriptionException(errorMessage);
27     }
28     else
29     {
30         throw new MQSubscriptionException("Respuesta inesperada del broker");
31     }
32 }
33 catch (MQClientException ex) when (!(ex is MQSubscriptionException))
34 {
35     throw new MQSubscriptionException($"Error al desuscribirse: {ex.Message}");
36 }
37 }

```

Listing 19: Implementación método Unsubscribe

III-B4. Requerimiento 004: *bool Publish(Message message, Topic topic)*

Envía la petición Publish al MQBroker mediante un socket de tipo cliente. El objeto Message encapsula el mensaje o contenido de la publicación e incluye cualquier funcionalidad necesaria para serializar/deserializar un mensaje como se recibió del socket. Retorna true o false indicando si el mensaje se pudo colocar o no. Debe lanzar excepciones relevantes según aplique.

IMPLEMENTACIÓN PARA EL REQUERIMIENTO 004 DEL MQCLIENT

El método Publish permite a un cliente publicar un mensaje en un tema específico del broker. Su implementación sigue un patrón similar a los métodos anteriores (Subscribe/Unsubscribe), pero con un enfoque en el manejo del contenido del mensaje y su entrega íntegra al broker. El flujo del método sigue la siguiente estructura:

1. Validación de parámetros:
 - Verifica que ni message ni topic sean nulos.
 - El constructor de Message ya maneja el caso de contenido nulo (lo convierte a string vacío).
2. Construcción del mensaje de publicación:
 - Formato: "PUBLISH—tema—contenido—appId".
3. Envío al broker mediante SendRequest:
 - Utiliza el mismo mecanismo de socket TCP con timeouts.
 - El contenido del mensaje se envía tal cual, sin serialización adicional.
4. Procesamiento de la respuesta:
 - OK—: Retorna true (publicación exitosa).
 - ERROR—: Lanza MQPublishException con el mensaje de error.
 - Respuestas inesperadas lanzan MQPublishException.

La implementación del código que maneja la publicación de mensajes en el servidor es el siguiente:

```

1
2 // Publica un mensaje en un tema específico
3 public bool Publish(Message message, Topic topic)
4 {

```

```

5     if (message == null)
6     {
7         throw new ArgumentNullException(nameof(message), "El mensaje no puede ser nulo");
8     }
9
10    if (topic == null)
11    {
12        throw new ArgumentNullException(nameof(topic), "El tema no puede ser nulo");
13    }
14
15    try
16    {
17        string request = $"PUBLISH|{topic.Name}|{message.Content}|{appId}";
18        string response = SendRequest(request);
19
20        if (response.StartsWith("OK|"))
21        {
22            return true;
23        }
24        else if (response.StartsWith("ERROR|"))
25        {
26            string errorMessage = response.Substring(6); // Skip "ERROR|"
27            throw new MQPublishException(errorMessage);
28        }
29        else
30        {
31            throw new MQPublishException("Respuesta inesperada del broker");
32        }
33    }
34    catch (MQClientException ex) when (!(ex is MQPublishException))
35    {
36        throw new MQPublishException($"Error al publicar: {ex.Message}");
37    }
38 }

```

Listing 20: Implementación método Publish

III-B5. Requerimiento 005: Message Receive(Topic topic)

Envía la petición Receive al MQBroker mediante un socket de tipo cliente. Retorna un objeto Message con el contenido del mensaje. Debe lanzar excepciones relevantes según aplique.

IMPLEMENTACIÓN PARA EL REQUERIMIENTO 005 DEL MQCLIENT

El método Receive permite a un cliente suscrito obtener el siguiente mensaje disponible para un tema específico del broker. Sigue un patrón de consumo FIFO (First-In-First-Out) y maneja cuidadosamente los casos donde no hay mensajes disponibles o ocurren errores en la comunicación.

El flujo del método implementado es el siguiente:

1. Validación del parámetro topic:
 - Rechaza topic nulo con ArgumentNullException.
 - La validación del nombre del tema ya está gestionada por la clase Topic.
2. Construcción del mensaje de solicitud:
 - Formato: RECEIVE—nombreTema—appId”.
3. Envío al broker mediante SendRequest:
 - Utiliza el mismo mecanismo de socket TCP con timeouts (5 segundos).
4. Procesamiento de la respuesta:

- OK—contenido: Retorna un nuevo Message con el contenido.
- ERROR—: Lanza MQReceiveException con el mensaje de error.
- Respuestas inesperadas lanzan MQReceiveException.

La implementación del código que maneja el método para recibir mensajes es el siguiente:

```

1
2 public Message Receive(Topic topic)
3 {
4     if (topic == null)
5     {
6         throw new ArgumentNullException(nameof(topic), "El tema no puede ser nulo");
7     }
8
9     try
10    {
11        string request = $"RECEIVE|{topic.Name}|{appId}";
12        string response = SendRequest(request);
13
14        if (response.StartsWith("OK|"))
15        {
16            // Extraer el contenido del mensaje
17            string content = response.Substring(3); // Skip "OK|"
18            return new Message(content);
19        }
20        else if (response.StartsWith("ERROR|"))
21        {
22            string errorMessage = response.Substring(6); // Skip "ERROR|"
23            throw new MQReceiveException(errorMessage);
24        }
25        else
26        {
27            throw new MQReceiveException("Respuesta inesperada del broker");
28        }
29    }
30    catch (MQClientException ex) when (!(ex is MQReceiveException))
31    {
32        throw new MQReceiveException($"Error al recibir: {ex.Message}");
33    }
34 }

```

Listing 21: Implementación método Receive

III-C. Tercera Parte: Interfaz Gráfica en .NET Core

La tercera parte es un programa con interfaz gráfica en .NET Core que permitirá probar las funcionalidades del sistema MQBroker. La interfaz gráfica puede ser similar a la siguiente (con las funcionalidades mínimas):

TEMA	CONTENIDO

Figura 1: Ejemplo visual de la interfaz gráfica con sus funciones básicas en .NET Core.

IMPLEMENTACIÓN DE LA INTERFAZ GRÁFICA EN .NET CORE

La interfaz gráfica desarrollada en .NET Core sirve como cliente de prueba para interactuar con el sistema MQBroker, ofreciendo una forma intuitiva y visual de probar todas las funcionalidades del sistema de mensajería. Su diseño prioriza la usabilidad y claridad por medio de botones, Textbox y ComboBox. La interfaz gráfica final tiene el siguiente aspecto:

The screenshot shows a Windows application titled "Message Queue Test Application". The interface is divided into several sections:

- Connection Settings:** Contains input fields for "MQ Broker IP:" (127.0.0.1), "MQ Broker Port:" (5000), and "ApplID:" (c317f647-b0ca-4). A blue "Connect" button is below these fields.
- Estado:** Displays "Desconectado".
- Publish Message:** Includes a "Tema:" dropdown menu, a large text area for "Contenido de la publicación:", and a blue "Publicar" button. A "Resultado:" label is at the bottom left of this section.
- Topic Subscription:** Features a "Tema:" dropdown menu, "Suscribirse" and "Desuscribirse" buttons, and a table with columns "Topic", "Status", and "Tema".
- Receive Messages:** Includes a "Tema:" dropdown menu, a "Recibir" button, and a table with columns "Topic", "Content", "Timestamp", and "Tema".
- Log:** A large text area at the bottom of the application.

Figura 2: Interfaz Gráfica implementada en .NET Core.

IV. DISEÑO GENERAL

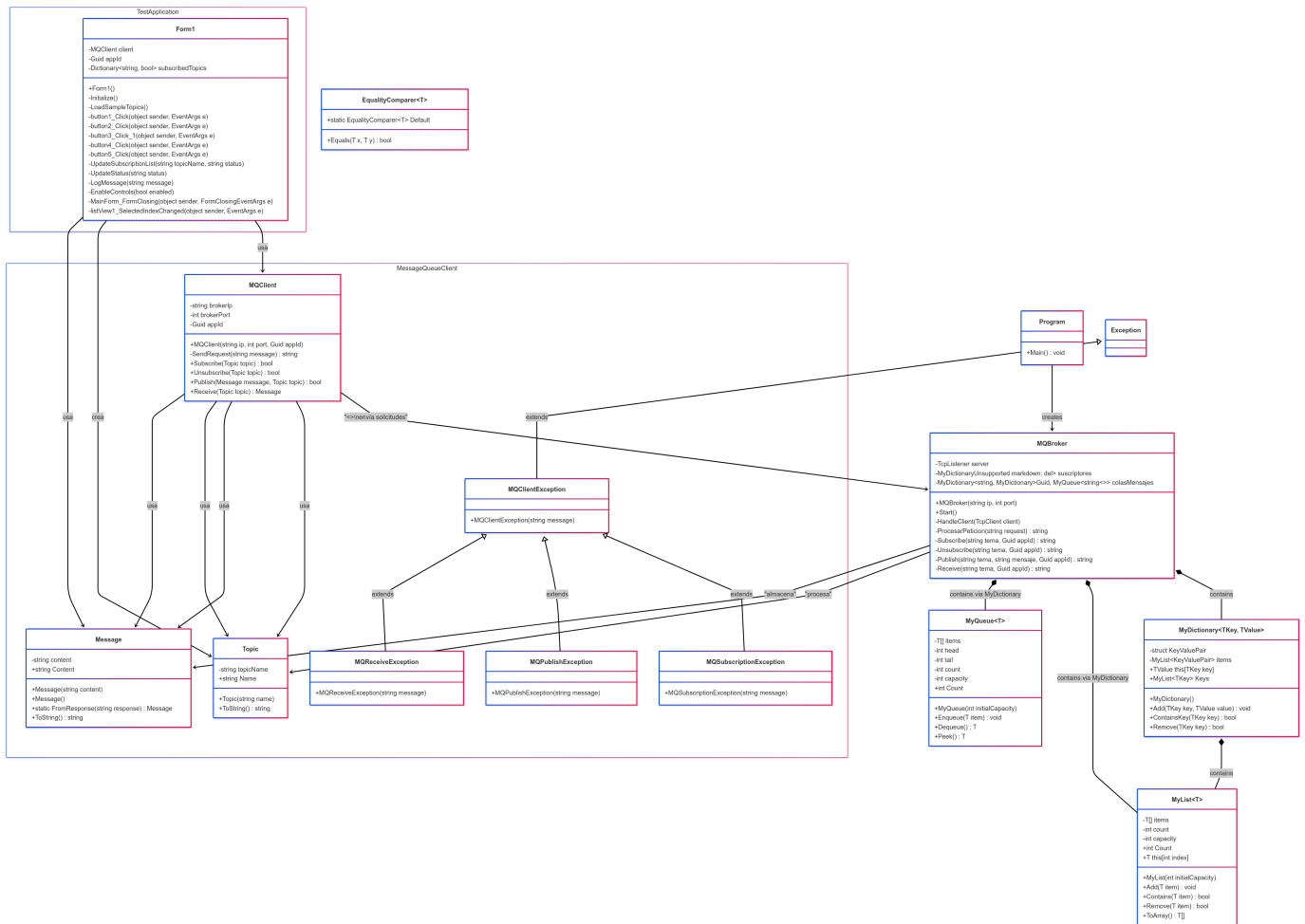


Figura 3: Diagrama UML del proyecto.

V. LINK AL REPOSITORIO DEL PROYECTO EN GITHUB

https://github.com/Sarimoca/Proyecto1_Datos1_S1_2025.git

REFERENCIAS

- [1] A. S. Tanenbaum and M. v. Steen, *Distributed systems: Principles and paradigms*. Pearson Prentice Hall, 2007.
- [2] D. Vohra, *Kubernetes microservices with Docker*. Apress, 2016.
- [3] W. R. Stevens and T. Narten, "Unix network programming," *ACM SIGCOMM Computer Communication Review*, vol. 20, no. 2, pp. 8–9, 1990.
- [4] R. T. Fielding, *Architectural styles and the design of network-based software architectures*. University of California, Irvine, 2000.
- [5] L. Bass, I. Weber, and L. Zhu, *DevOps: A software architect's perspective*. Addison-Wesley Professional, 2015.